

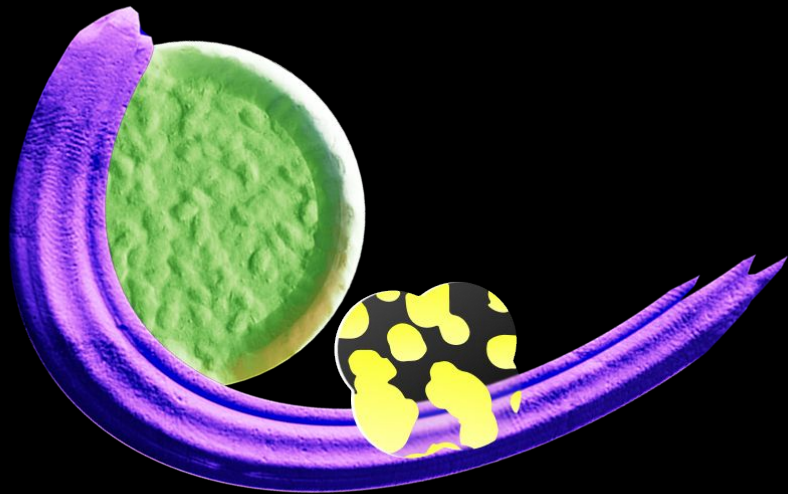
Отметьте студентов в журнале

Domain-Driven Design – проектирование вокруг предметной области



Михаил Каморин
Tech Lead

Speaker

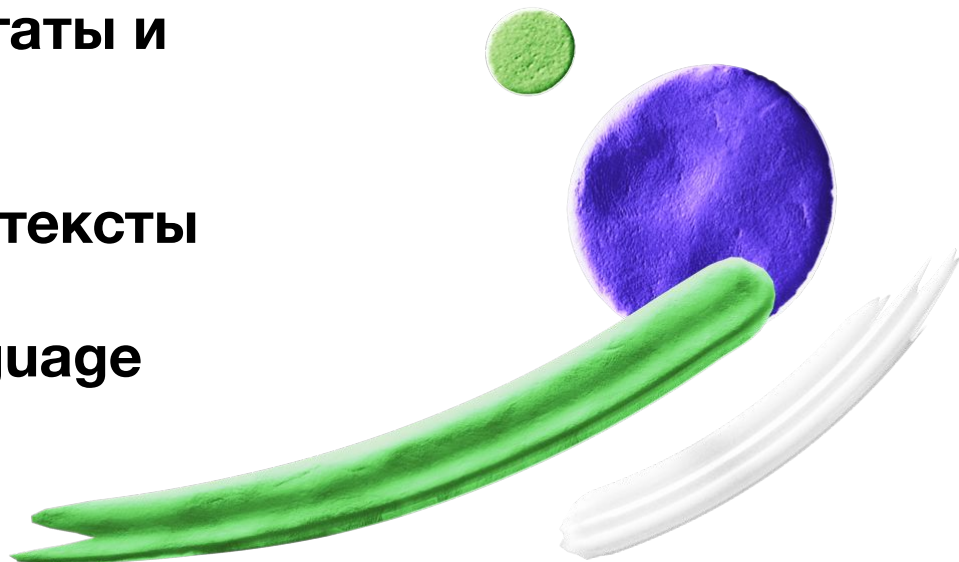


Михаил Каморин
Tech Lead

16 лет в IT, 6 лет вёл
разные курсы в OTUS

Сегодня вы узнаете, как:

- 1 Выделять сущности, агрегаты и value-объекты
- 2 Разделять систему на контексты
- 3 Применять ubiquitous language



План занятия

- 1 Погружение в предметную область
- 2 Bounded Context и контекстные границы
- 3 Ubiquitous Language – единый язык модели
- 4 Архитектурные границы, отражающие бизнес

I. Погружение в предметную область

01 Анализ требований и выявление
доменных понятий

02 Сущности, агрегаты и value-объекты



Предметная область

- **Предметная область** – реальность, в которой живёт бизнес
- **Ключевая идея:** сначала понять, что нужно бизнесу, потом решать, как это реализовать
- Анализ требований опирается на активную коммуникацию с экспертами в доменной области (domain experts)

Инструменты анализа требований

- **Интервью** – обсуждаем сценарии и исключения
- **User stories и сценарии** – подробно разберём в следующий раз
- **Event Storming** – коллективное моделирование событий

Пример интервью

- **Процесс:** формирование заказа в интернет-магазине
- **Вопросы:**
 - Что происходит, когда заказ оформляется?
 - Может ли клиент оплатить заказ позже?
 - Что делает система, если товар закончился на складе?

Пример Event Storming

- **Процесс:** выполнение заказа в интернет-магазине
- **События:**

Заказ создан

Платёж получен

Товар отправлен

Доставка завершена

Сущности

- **Сущность (Entity)** – объект с устойчивой идентичностью
- Изменение параметров сущности не изменяет идентичность
- Как правило, идентичность определяется уникальным идентификатором, часто суррогатным

Value-объекты

- **Value-объект (Value Object)** – просто значение, не обладает идентичностью, только состоянием
- Должны быть иммутабельными
- Если значение двух value-объектов совпадает, то это один и тот же value-объект

Агрегаты

- **Агрегат (Aggregate)** – группа сущностей и value-объектов, которые изменяются как единое целое
- Агрегаты помогают ограничить **зону консистентности**, гарантируют согласованность своего содержимого

Пример “Управление заказом”

- **Сущности** – Order, Customer, Payment
- **Value-объекты** – Money, Address
- **Агрегат** – Order (включает позиции заказа, оплату, статус и пр.)
 - `Order.create()` создаёт заказ
 - `Order.markAsPaid()` меняет статус на “оплачен”
 - `Order.cancel()` отменяет заказ

Вопрос для общего обсуждения

Приведите пример, когда ФИО человека будет сущностью, а когда value-объектом

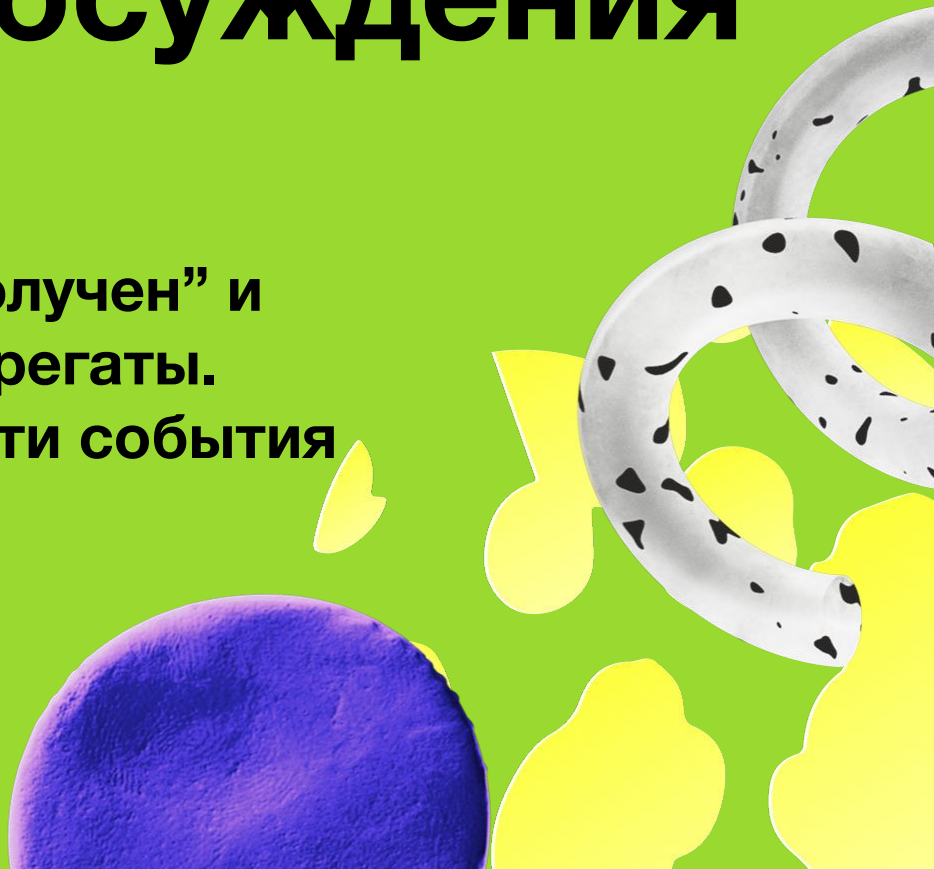
2 минуты



Вопрос для обсуждения в парах

**Возьмите события “Платёж получен” и
“Заказ оплачен” и опишите агрегаты.
Обсудите, можно ли считать эти события
идентичными?**

3 минуты



II. Bounded Context и контекстные границы

- 01 Что такое Bounded Context
- 02 Различие моделей в разных контекстах
- 03 Контекстные карты (Context Maps)



Bounded Context

- **Bounded Context** – граница смыслов и моделей
 - Термины внутри контекста имеют конкретный смысл
 - Язык соседних контекстов может переиспользовать термины с другим смыслом
 - Содержит собственную модель предметной области, адаптированную к своим задачам

Границы контекстов

- Разное понимание терминов
- Разные источники правды (кто отвечает за состояние данных)
- Процессы и команды отделены
- Разные циклы изменений

Различие моделей в разных контекстах

- Модели в разных контекстах могут конфликтовать
- **Пример:**
 - В “Продажах” заказ может быть “создан”, “оплачен”, “доставлен”
 - В “Складе” он может быть “в обработке”, “в сборке”, “отгружен”
- Модель должна быть верна внутри контекста, а не во всей компании

Контекстные карты

- **Контекстная карта (Context Map)** описывает
 - какие контексты есть в системе;
 - как они связаны между собой;
 - кто от кого зависит;
 - как устроены связи

Customer / Supplier

- Контекст 1 (поставщик, Supplier) предоставляет данные
- Контекст 2 (потребитель, Customer) их использует
- **Пример:**
 - “Каталог” (Supplier) предоставляет товары для “Заказы”

Conformist

- Вариант **Customer / Supplier**
- Потребитель полностью подстраивается под модель поставщика
- **Пример:**
 - Поставщик – API внешнего сервиса (например, Госуслуги)

Anti-Corruption Layer

- Вариант **Customer / Supplier**
- Потребитель не хочет / не может использовать модель поставщика
- Со стороны потребителя появляется адаптер, который “переводит” данные на язык модели потребителя

Shared Kernel

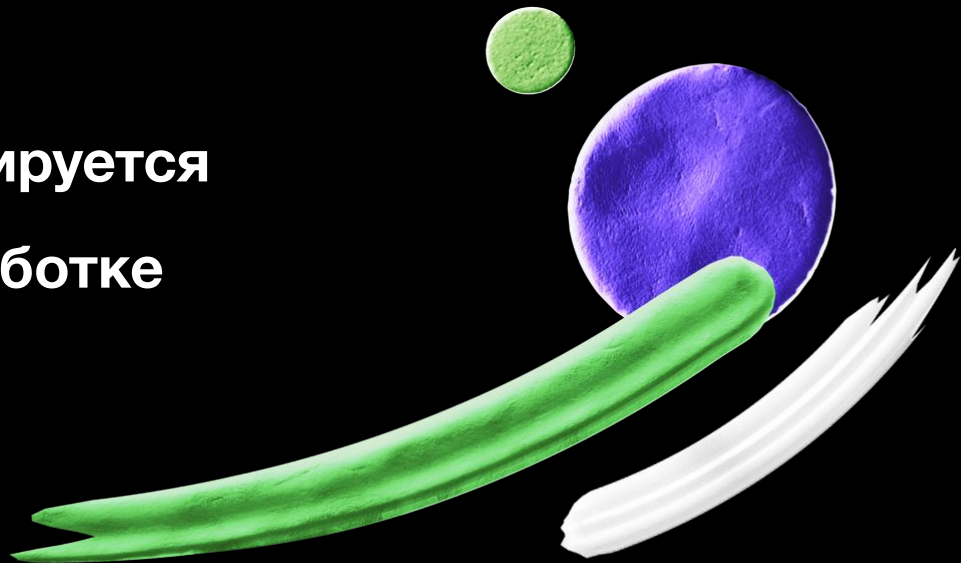
- Взаимодействующие контексты выделяют общую часть модели и поддерживают её совместно
- **Пример:**
 - Общие справочники (страны, валюты и пр.)
- **Ключевая идея:** ядро должно быть как можно меньше и включать в себя только интеграционные контракты

Кейс

- **Кейс:** Интернет-магазин. Выделите 3-4 контекста. Приведите пример конфликта моделей (не из лекции). Опишите 3-4 связи между контекстами
- **Формат:** делимся на три группы
- **Время:** 6 минут + по 1 минуте на 1 контекст от каждой группы
- **Результат:** мини-презентация

III. Ubiquitous Language – единый язык между бизнесом и разработкой

- 01 Что это такое и как формируется
- 02 Использование при разработке
- 03 Антипаттерны и ловушки



Ubiquitous Language

- **Ubiquitous Language** – единый словарь понятий, который одинаково понимают и бизнес, и разработчики
- Язык не просто описывает систему, он становится кодом
- Убирается барьер между “техническим” и “бизнесовым” мышлением

Как формируется

- **Интервью** с доменным экспертом
- Важно не просто записать ответы, а зафиксировать контекст
- **Пример:** слово “платёж” может означать разные вещи
 - инициирование денежного перевода
 - факт списания денег
 - подтверждение успешной транзакции

Использование в разработке

- Названия классов, методов, событий, полей должны использовать те же термины, что и бизнес
- “Разместить заказ” = ~~CreateOrder~~PlaceOrder
- “Счёт оплачен” = ~~PaymentApprovedEvent~~InvoiceSettledEvent
- “Покупатель” (не любой пользователь) = ~~User~~Customer

Документирование через язык

- **Event Storming** – описание всех событий, которые происходят в системе
- Событие – глагол, состояние – существительное
- Диаграммы и схемы также используют единый язык

Антипаттерны

- Язык только в голове архитектора. Разработчики не вовлечены, и язык существует только в документации
- Технический жаргон вместо бизнес-языка. Например, `OrderReadyForProcessingFlag`
- Слишком общий язык. Все контексты пытаются договориться об одной универсальной модели.

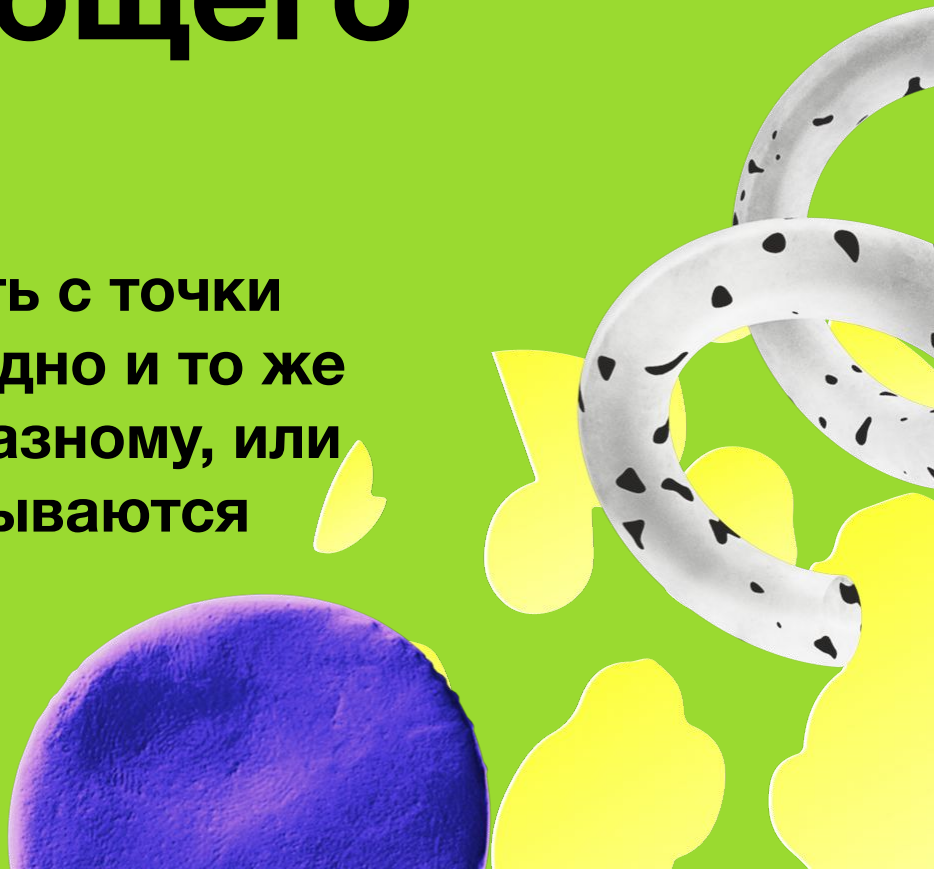
Живая архитектура

- Изменился процесс → поменялись слова → поменялась модель → поменялся код
- Единый язык согласует смысл:
 - Создаётся в диалоге
 - Живёт в коде
 - Поддерживается всей командой

Вопрос для общего обсуждения

Какую проблему проще решить с точки зрения Ubiquitous Language: одно и то же событие, называющееся по-разному, или разные события, которые называются одинаково?

2 минуты



IV. Архитектурные границы, отражающие бизнес

- 01 Соответствие архитектуры бизнес-циклам
- 02 Наложение контекстов на архитектуру
- 03 Взаимодействие контекстов



Соответствие архитектуры бизнес-циклам

- Архитектура должна повторять форму бизнеса
- Автономные бизнес-направления должны приводить к модульной архитектуре
- Автономные контексты: изменения в одном контексте не должны ломать другие

Наложение контекстов на архитектуру

- Контексты буквально воплощаются физически
- **Сервис** – реализует один Bounded Context, управляет своими данными, отвечает за бизнес-правила
- **Модуль** – реализует один Bounded Context, может быть частью монолита
- **Интеграционный канал** – события, API, сообщения (способы взаимодействия между контекстами)

Взаимодействие контекстов

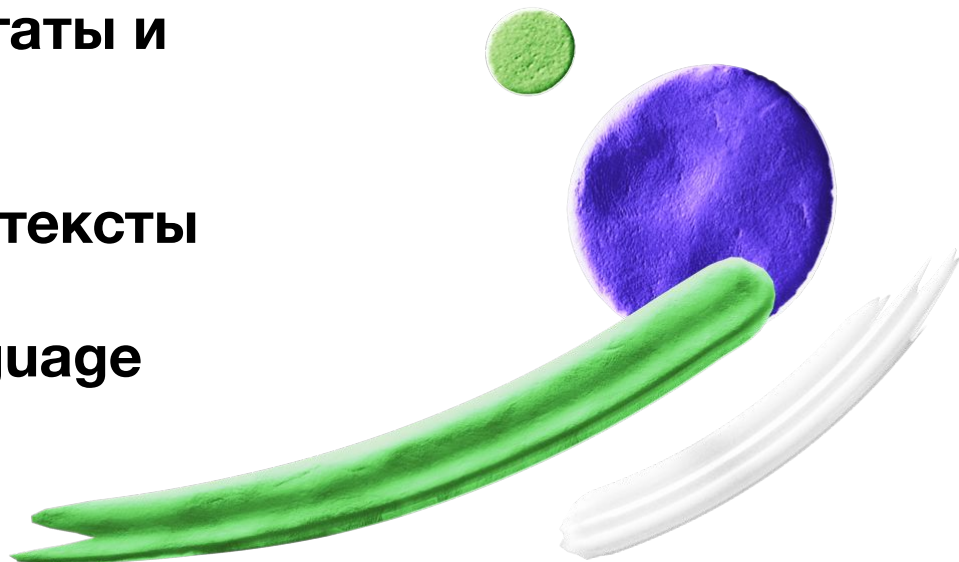
- Стратегия интеграции должна определяться бизнес-ограничениями, рисками и частотой изменения
- **Примеры:**
 - Anti-Corruption Layer позволяет изолировать частые изменения
 - Shared Kernel подходит для тесно связанных контекстов с редкими изменениями

Кейс

- **Кейс:** Система управления заказами. Требуется разделить домен на контексты, определить границы, выбрать способы интеграции между контекстами, зафиксировать язык
- **Формат:** делимся на три группы
- **Время:** 10 минут + по 2 минуте на мини-презентацию от каждой группы
- **Результат:** мини-презентация

Сегодня мы узнали, как:

- 1 Выделять сущности, агрегаты и value-объекты
- 2 Разделять систему на контексты
- 3 Применять ubiquitous language



Вопросы?